Ders 8 Genişletilmiş Notu: LZ77, LZ78, LZW ve Görüntü Sıkıştırma (PNG, Faks, JPEG, GIF, TIFF)

Ders 8 Genişletilmiş Notu: LZ77, LZ78, LZW ve Görüntü Sıkıştırma (PNG, Faks, JPEG, GIF, TIFF)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ders Başındaki Sorular: Huffman'da İkili Karakterler ve Görüntüde Huffman
- 2. Önceki Yöntemlerin Kısa Tekrarı
- 3. LZ ve Türev Algoritmalar
- 4. LZ77: Kayan Pencere
- 5. LZ77 Token'ı ve Offset Seçimi
- 6. "sir sid eastman" Örneği: Kodlama ve Kod Çözme
- 7. Token Uzunluğu ve Kazanç
- 8. LZ77'nin Özellikleri ve Dairesel Kuyruk
- 9. LZ78
- 10. Trie Ağacı ve Sözlüğün Dolması
- 11. LZW
- 12. Görüntü Sıkıştırmaya Giriş ve PNG
- 13. Run-Length, Faks Standartları ve Arşivleyiciler
- 14. JPEG-LS Tahmini
- 15. JPEG
- 16. JPEG 2000, GIF, TIFF ve PNG
- Örnek Sorular

1. Ders Başındaki Sorular: Huffman'da İkili Karakterler ve Görüntüde Huffman

Ders, Huffman ile ilgili birkaç soruyla başladı. Bu soruların akademik kısmı, Huffman'ın tek karakterlerin dışında nasıl kullanılabileceğini gösterir.

Soru: Huffman ile doküman sıkıştırırken semboller tekli ve ikili karakterlerden oluşacaksa, ikili karakterlerin istatistiği çıkarılıp bir eşiğin üstündekiler mi alınmalı? **Cevap:** Derste Huffman anlatılırken her zaman tek karaktere bir sembol değeri verilmişti. İngilizcede "th" gibi, Türkçede de çok sık geçen ikili karakterler tek bir karaktermiş gibi aynı ağaca yerleştirilerek kod verilebilir. Önce metin okunur, bütün ikililerin frekansları çıkarılır ve en çok geçenler alınır. Ağaçta İngilizce için 26 karakterin her birinin mutlaka bir kodu olur, bunlara seçilen ikililerin kodları eklenir. Eşik çok yukarıda tutulursa daha az ikili alınır. Ağaç ne kadar büyürse üzerinde hareket etmek o kadar zorlaşır; birden fazla ağaç kullanmak algoritmayı çok karmaşık hale getirir. En çok kullanılan 3-4 ikiliyi eklemek yeterlidir. Kodlarken önce metinden iki karakter okunur; bu ikilinin çıkarılmış bir kodu varsa o kullanılır, yoksa tek karakter olarak okunup kodlanır.

Soru: Bir resmi Huffman ile kodlarken ne yapılmalı? **Cevap:** Mantık aynıdır; karakterler yerine resimdeki piksel renk değerlerine kod verilir. True color bir resim alınırsa Red, Green ve Blue kanallarına ayrı ayrı uygulamak gerekir. 256 renkli, yani color table kullanan bir resimde

ağaçta da 256 renk olacağı için 256 kod oluşur. Bu yüzden resim önce 256 renge indirgenip sonra sıkıştırılabilir. Giriş olarak Bitmap almak okumayı kolaylaştırır; JPEG gibi formatlar zaten sıkıştırılmıştır, onları çözmekle uğraşmanın anlamı yoktur.

Soru: Metindeki satır başı (carriage return), tab gibi görünmeyen karakterler ihmal edilebilir mi? **Cevap:** Kayıpsız sıkıştırma yapılıyorsa açılan verinin orijinalle aynı olması gerekir; aksi halde sıkıştırma kayıplı olur. Bu yüzden bu karakterler de birer sembol olarak tutulup kodlanmalıdır. Hoca, Türkçe için 29 harf, noktalama işaretleri ve birkaç ikili ile toplam sembol sayısının kabaca 45'i geçmeyeceğini, önemli olanın dosya boyutu değil algoritmanın çalışması olduğunu söyledi.

2. Önceki Yöntemlerin Kısa Tekrarı

Hoca, sözlük yöntemlerine geçmeden önce dönem boyunca görülen yöntemleri özetledi.

- **Ad hoc yöntemler:** Ad hoc ağlar gibi, yalnız uygulamanın türüne ve eldeki verinin içeriğine özgü geliştirilmiş algoritmalardır.
- **İstatistiksel yöntemler:** Bilginin istatistiğini, yani her karakterin, sembolün ya da string'in olasılık ya da frekans değerini kullanır. Önemli olan, sembolün metinde kaç kez karşımıza çıktığıdır.

İstatistiksel yöntemlerde iki önemli kavram vardı. **Entropi**, eldeki kaynağı kodlamak için ortalama kaç bite ihtiyaç olduğunu verir. Entropi 2,5 çıktıysa, semboller bazen 1-2 bitle, bazen 3-4-5 bitle kodlanabilir, ama ortalaması 2,5 olur. **Ortalama mesaj uzunluğu**, bir sembolü kodlamak için harcanan bit sayısının o sembolün olasılığıyla çarpılıp bütün semboller için toplanmasıyla bulunur. Amaç **redundancy**'yi, yani entropi ile ortalama mesaj uzunluğu arasındaki (mutlak) farkı düşürmektir; redundancy ne kadar küçükse seçilen yöntem o kadar iyidir.

Prefix kod çok önemli bir kavramdı: bir sembolün kodu, başka bir sembolün kodunun ön ekini oluşturmaz. Sıkıştırma kayıplı ya da kayıpsız olsun, geri dönüşüm ancak kodlar prefix özelliğine sahipse mümkündür.

Huffman'ın statik, semi-adaptif ve adaptif (dinamik) türleri görüldü. Huffman ağacı hiçbir zaman dengeli olmadığından, bazı prefix kodları oluşmadan ağacın derinliği artar ve başlık bilgisi daha fazla yer tutar. **Canonical Huffman**, önce sembollerin Huffman kodları çıkarıldıktan sonra kod uzunluklarını ve her uzunlukta kaç sembol olduğunu kullanarak başlıkta gönderilecek bilgiyi azaltır. Burada "sıkıştırmak" bir algoritma uygulamak değil, mevcut prefix kodları daha az bilgiyle göndermektir.

Son olarak karakter yerine string sıkıştırmanın etkisine bakılmıştı. Önceki dersteki iki sembollü örnekte üçlü string'lerle sembol başına ortalama uzunluk yaklaşık 0,72 bite iner; string 5-6 karaktere çıkarılırsa bu değer daha da küçülür. Buradan "sözlük kullanarak sıkıştırabilir miyiz?" sorusuna geçilir.

3. LZ ve Türev Algoritmalar

Sözlük kullanarak sıkıştırma yapan algoritmaların en önemlileri LZ ve türevleridir. İyi sıkıştırma verirler ama süreleri daha uzun olabilir. Hoca yalnız temel olanları anlatacağını, diğer türevlerin bu algoritmaların bazı parametrik değerleriyle oynadığını söyledi.

Slayttaki bilgiler şunlardır (ekrandan, 00:20:00): Jacob Ziv ve Abraham Lempel isimli iki araştırmacı 1970'li yıllarda LZ77 ve LZ78 adını verdikleri ilk sözlük tabanlı sıkıştırma algoritmasını geliştirdi. Adın Z'si Ziv'den, L'si Lempel'den gelir; LZ77 adı 1977 yılından gelir. Bu çalışma referans alınarak benzer yöntemler geliştirildi:

```
LZSS  -> Storer-Szymanski
LZFG  -> Fiala-Greene
LZRW1 -> Ross-Williams
LZW   -> Welch
LZMW  -> Miller-Wegman
LZAP  -> (All prefix)
LZY   -> (Yabba)
LZP   -> (Prediction)
```

Buradaki “sözlük” kavramı aklı karıştırmamalıdır. Bu, bir dilin kurumsal sözlüğü ya da imla kılavuzundaki gibi sıralanmış kelimeler değildir. Belirli string’lerden oluşur; bazıları anlamlı bir kelime olabilir, bazıları olmayabilir. Sözlük niteliğinde olduğu için bu algoritmalara sözlük tabanlı denir.

Hoca, LZ algoritmalarının en iyi versiyonunun Welch’in katkısıyla ortaya çıkan LZW olduğunu söyledi.

> **Ders sonrası not:** Hoca LZW’nin WinRAR, PKZip gibi uygulamalarda kullanıldığını söyledi. Dersin ilerisinde kendisi de belirttiği gibi ZIP ve gzip, LZ77 türevi ile Huffman’ı (DEFLATE) kullanır; WinRAR’ın yöntemi de büyük olasılıkla LZ77 ailesindendir. LZW’nin bilinen kullanımları GIF, TIFF ve Unix compress’tir (büyük olasılıkla).

4. LZ77: Kayan Pencere

4.1 Ana fikir

LZ77 literatürde **sliding window** (kayan pencere) olarak geçer. Algoritmanın ana fikri, input stream’den önceden okunmuş bilgiyi sözlük gibi kullanmaktır. Input stream’den veri gelir, işlenir ve sıkıştırılmış hali output stream’e atılır; işlenmiş her veri sözlük olarak kullanılır.

Kodlayıcı input stream için bir pencere oluşturur ve kodlanan değerleri bu pencerede sağdan sola öteler. Veri sağdan gelir, işlenir, işlendiği yer sözlük olur. Bu sözlüğün de bir sınırı vardır, sonsuz kaynak yoktur; sınır dolunca ilk giren sembol pencereden çıkar.

4.2 Pencerenin iki parçası

- **Arama alanı (search buffer):** Pencerenin solundaki bölümdür. En son okunan ve kodlanan sembollerden oluşur. Burası aynı zamanda sözlük alanıdır.
- **Kaynak kod alanı (look-ahead buffer):** Pencerenin sağındaki, input stream’in aktığı bölümdür. Burada henüz kodlanmamış semboller vardır.

Pratikte arama alanı binlerce byte uzunluğundayken kaynak kod alanı onlarca byte uzunluğundadır; arama alanı her zaman daha büyüktür. Bu boylar parametriktir. Bazı türevler bu iki alanın dengede olmasının daha iyi olduğunu söyler, bazıları token’ları farklı yerleştirir. Bütün bunlar farklı türevlerin çıkmasına neden olur; sonuçta yapılan iş aynıdır.

> **Ders sonrası not:** Hoca arama alanının dolma mantığını bir yerde “ilk giren son çıkar” diye söyledi. Anlattığı işleyiş ve dersin ilerisinde kendi söylediği “ilk giren ilk çıkar” mantığıdır (FIFO).

4.3 Slayttaki örnek (ekrandan, 00:30:00)

Slaytta pencere şöyledir:

```
<-- coded text  | sir sid eastman easily t | eases sea sick seals |  text to be read <--
                |      arama alanı         |   kaynak kod alanı   |
```

Başlangıçta arama alanı her zaman boştur; input’tan gelen veriler işlendikçe buraya atılır. Algoritmanın bir süre çalıştığını ve sıranın kaynak kod alanının ilk sembolü olan “e”ye geldiğini düşünelim.

- Kodlayıcı “e”yi arama alanında sağdan sola aramaya başlar.
- İlk “e”yi “easily” kelimesinde yakalar. Slayta göre bu “e”nin offset değeri 8’dir.
- Daha uzun bir string yakalanabilir mi diye bakılır: ikinci sembol “a” da, üçüncü sembol “s” de eşleşir. Dördüncü sembolde (“e” ile “i”) eşleşme biter. Üçlü eşleşme bulunmuştur.
- Ama arama bitmez. Arama alanının devamında “eas” ile başlayan başka bir string var mı diye bakılır. Kodlayıcı “e”yi ikinci kez “eastman” kelimesinde yakalar; “eas” orada da eşleşir, dördüncü sembol eşleşmez.
- Kodlayıcı en uzun çakışmayı bulduktan sonra, en son bulunan çakışmayı referans alarak **(16, 3, “e”)** token’ını hazırlar.

Hoca konumları sayarken önce “burası sıfırıncı indis” dedi, sonra 1’den başlayarak saydı: boşluk 1, y 2, l 3, i 4, s 5, a 6, e… şeklinde giderek ilk “e” için 8’i, devamında ikinci “e” için 16’yı

buldu. 16 offset'e, 3 çakışan sembol sayısına, "e" de çakışmadan sonra kaynak kod alanında gelen sembole karşılık gelir.

Token output stream'e yazıldıktan sonra pencere kayar: "eas" ve arkasındaki "e" arama alanına geçer, arama alanının solundan 4 sembol dışarı çıkar ve input stream'den 4 yeni sembol kaynak kod alanına alınır.

5. LZ77 Token'ı ve Offset Seçimi

5.1 Üç alanlı token

LZ77'de token üç bilgiden oluşur:

- Çakışmanın offset'i,
- Kaç sembolün çakıştığı (length),
- Çakışmadan sonra gelen sembol.

Soru: Üç sembolü eşleştirdik, "e"yi eşleştiremedik; neden onu da token'a koyuyoruz? **Cevap:** Algoritmayı ortaya atan kişi token'ı bu şekilde üç alanlı tanımlamıştır. Bunun nedeni çakışma olmayan durumlarda ortaya çıkar (bkz. 6.1): başlangıçta hiçbir çakışma olmadığı için sembol üçüncü alana yazılmazsa algoritma ilerleyemez. Bazı türevler token'ı iki alanlı yapar ve bu sembolü token'a katmaz; türevlerin çıkma nedeni de tam olarak bu tür "acaba daha başarılı olur mu" denemeleridir. Aynı araştırmacıların 1977 ve 1978'de yaptıkları algoritmalar da birbirinden çok farklıdır.

Geriye doğru hiç çakışma yoksa offset yerine 0 yazılır; çakışan sembol sayısı da 0'dır. Başlangıçta sözlük alanı boş olduğu için offset'ler her zaman 0 olur.

5.2 En yakındaki mi, en uzaktaki mi?

Aynı uzunlukta iki eşleşme varsa hangisinin offset'i kullanılmalıdır? Bir yaklaşım "ilk bulduğunu kullan, daha geriye gitme" der; diğeri "en son (en uzakta) bulduğunu kullan" der. İkisinin de avantajı vardır:

- **En uzaktaki çakışma:** Token'ı oluşturmak kolaydır, algoritma basitleşir. Ancak en uzaktaki eşleşmenin offset'i her zaman ilkinden büyüktür; offset'i tutmak için daha fazla bit gerekir.
- **En yakındaki çakışma:** Offset küçüktür. Ama bağlı listede olduğu gibi geriye dönüp ilk eşleşmeyi bulmak gerekir; aynı string 10 kez tekrar ediyorsa 10 kez geriye gidilir.

Yani ya algoritma karmaşıklaşır ama offset daha az bitle tutulur, ya da offset büyür ama algoritma basitleşir. Bu da farklı türevlerin doğmasına neden olmuştur. Derste gösterilen LZ77 en uzaktaki (sözlüğün sınırına kadar aranıp en son bulunan) çakışmayı kullanır.

6. "sir sid eastman" Örneği: Kodlama ve Kod Çözme

6.1 Kodlama (ekrandan, 00:40:00 ve 00:46:10)

Slayttaki tablo, arama alanı boşken başlayan kodlamayı gösterir (␣ boşluktur):

```
arama alanı | kaynak kod alanı      token
            | sir␣sid␣eastman␣      (0,0,"s")
          s | ir␣sid␣eastman␣e      (0,0,"i")
         si | r␣sid␣eastman␣ea      (0,0,"r")
        sir | ␣sid␣eastman␣eas      (0,0,"␣")
       sir␣ | sid␣eastman␣easi      (4,2,"d")
```

Adım adım:

- "s" arama alanında yoktur (alan boştur). Offset 0, çakışma 0; sembolün kendisi "s" üçüncü alana yazılır. Hoca burada üçüncü alanın neden gerektiğini gösterdi: hiç çakışma olmadığı halde "s" yazılmazsa geri dönüş olamaz, ilk karakterin ne olduğu hiçbir zaman bulunamaz.
- "s" sözlüğe geçer, pencere bir kayar ve input'tan yeni bir karakter ("e") eklenir. "i" sözlükte yoktur: (0, 0, "i").

- “r” yoktur: (0, 0, “r”). Boşluk yoktur: (0, 0, “␣”).
- Sıra “s”ye gelir; “s” sözlükte vardır. Amaç hep en uzun eşleşmeyi bulmaktır: “s”den sonra “i” de vardır, ama “d” yoktur. “s” sağdan 4. konumdadır, 2 sembol çakışmıştır, çakışmayan sembol “d”dir: (4, 2, “d”).

Hoca tabloyu kalemle devam ettirdi (ekrandan, 00:46:10). Arama alanında artık “sir␣sid” vardır:

- Boşluk sözlükte vardır (sağdan 4. konum), ama arkasından “e” gelmez. Bir çakışma vardır: **(4, 1, “e”)**.
- “a” sözlükte yoktur: **(0, 0, “a”)**.
- “s” iki yerde vardır, ama hiçbirinin arkasından “t” gelmez. En uzaktaki 10. konumdur: **(10, 1, “t”)**.
- “m” yoktur: **(0, 0, “m”)**.
- “a” yalnız bir yerde vardır (4. konum), arkasından “n” gelmez: **(4, 1, “n”)**.

Bu token’lar, en uzaktaki eşleşmeyi seçen bir LZ77 kodlayıcısı python ile yazılarak doğrulandı: (0,0,s) (0,0,i) (0,0,r) (0,0,␣) (4,2,d) (4,1,e) (0,0,a) (10,1,t) (0,0,m) (4,1,n).

6.2 Kod çözme

Soru (hocanın kendi sorusu): Output stream’e yalnız bu token’lar yazıldı; orijinale nasıl dönülür? **Cevap:**

- (0, 0, “s”): çakışma yoktur, “s” yazılır.
- (0, 0, “i”), (0, 0, “r”), (0, 0, “␣”): sırayla “i”, “r” ve boşluk yazılır. Elde “sir␣” vardır.
- (4, 2, “d”): 4 sembol geri gidilir, orada “s” vardır; 2 sembol (“si”) alınır, arkasına “d” eklenir. Elde “sir␣sid” olur.
- (4, 1, “e”): 4 geri gidilir, boşluk vardır; 1 sembol alınır, arkasına “e” eklenir.
- (0, 0, “a”): “a” eklenir.
- (10, 1, “t”): 10 geri gidilir, “s” vardır; 1 sembol alınır, arkasına “t” eklenir. Elde “sir␣sid␣east” olur.

Görüldüğü gibi açma çok kolaydır, hiçbir problem çıkmaz.

7. Token Uzunluğu ve Kazanç

Metin uzadıkça sıfır offset’ler azalır. Dokümanlarda aynı kelimeler defalarca geçer; örneğin “eastman” başka bir yerde tekrar geçtiğinde çok daha uzun bir eşleşme olur. Token’lar ilerledikçe sıkıştırma performansı artar. Bir karakter tek byte (8 bit) ile kodlanabilirken token offset, length ve sembol için 8 bitten fazla yer tutar; bu yüzden ilk başlarda sıkıştırma sağlanmaz, ama doküman ilerledikçe performans yükselir.

Slayttaki kurallar şunlardır (ekrandan, 00:40:00):

- Search buffer’ın uzunluğu S ise offset’in uzunluğu $\lceil \log_2 S \rceil$ olur.
- Pratikte offset’in uzunluğu 10-12 bit olur.
- Look-ahead buffer’ın uzunluğu L ise length alanının uzunluğu $\lceil \log_2(L-1) \rceil$ olur.
- Pratikte look-ahead buffer birkaç on byte’tan oluşuyorsa length alanı birkaç bit olur.
- Sembol alanı 8 bittir. Yaklaşık token uzunluğu $11 + 5 + 8 = 24$ bittir.

Hoca bunu şöyle yorumladı: iki çakışma ve bir sembol içeren bir token 3 karakteri kodlar; bu 3 karakter orijinalde $3 \times 8 = 24$ bit tutar, token da 24 bittir. Daha uzun eşleşmelerde artık kazanç başlar. LZ türevlerinde araştırmacıların yaptığı da budur: offset’in boyu, length’in boyu, token’a ayrılan yer ve sembolün token’ın içinde mi ayrı mı verileceği değiştirilir.

> **Ders sonrası not:** Hoca arama alanının pratik boyunu anlatırken bir yerde “binlerce byte”, slaytı okurken “birkaç byte, 3 byte, 4 byte” dedi. Offset 10-12 bit ise $S = 2^{10}$ ile 2^{12}, yani 1024-4096 byte’tır; arama alanı büyük olasılıkla birkaç kilobyte’tır.

8. LZ77'nin Özellikleri ve Dairesel Kuyruk

8.1 Kod çözücü daha basittir

Kodlarken offset sayma, geriye dönme gibi işler vardır; kod çözmek kodlamaktan daha kolaydır. Kod çözücüde kodlayıcının pencere boyuna eşit bir buffer alanı oluşturulur; buradaki "window" aslında bellekte işlem yapılan alandır. Kod çözücü token'ı okur, buffer'ında çakışan sembolleri bulur, bunları ve üçüncü alanı output stream'e yazar ve buffer alanını öteler.

Bu yöntem arşiv uygulamalarına elverişlidir. Arşive bir kez yazılan bilginin sürekli olarak arşivden çekilmesi gerekebilir; bu yüzden arşiv uygulamalarında kod çözmenin basit olması istenir.

Örüntüler birbirine yakın tekrar ediyorsa performans da iyi olur. Hoca Türkçeden örnek verdi: "ka", "la", "na", "ne" gibi ikili heceler kelimelerin içinde çok sık geçer; bunlar sıkıştırma performansını artırır.

8.2 Türevler ve dairesel kuyruk (ekrandan, 01:00:00)

LZ türevleri offset ve length alanlarıyla oynar, buffer'ların boyutlarını değiştirir ve kayan pencere mantığında değişiklik yapar. Derste pencere hep doğrusal okundu: veri sağdan gelir, soldan çıkar, ilk giren ilk çıkar. Bazı versiyonlarda ise **dairesel kuyruk (circular queue)** kullanılır. Slayttaki kurallar:

- Bu yöntemde "s" (start) ve "e" (end) olmak üzere iki pointer vardır.
- Karakterler end pointer'dan eklenir, start pointer'dan çıkarılır.
- Dairesel kuyruğa yeni semboller eklendikçe pointer'lar yer değiştirir.

Slayttaki şekil şu adımları gösterir:

```
(a) [sid␣east        ]   s başta, e "east" sonunda
(b) [sid␣eastman␣easi]   s başta, e sonda (kuyruk dolu)
(c) [lid␣eastman␣easi]   "l" en eski "s"nin yerine yazıldı; e ve s yan yana
(d) [lid␣eastman␣easi]   e ve s pointer'ları
(e) [ly␣␣eastman␣easi]   e ve s yan yana
(f) [ly␣teastman␣easi]   e ve s yan yana
```

Doğrusal pencerede yeni bir "k" sembolü gelecekse en soldaki sembol dışarı atılır, diğerleri birer kayar ve "k" sona yerleştirilir; start ve end yine aynı noktalardadır. Dairesel yapıda ise dizinin sonu başına bağlanmış gibi düşünülür. Kuyruk doluyken "l" gelince start'ın gösterdiği en eski sembol ("s") atılır ve "l" onun yerine yazılır; end pointer artık "l"yi, start pointer bir sonraki en eski sembolü ("i") gösterir. Hiçbir sembol kaydırılmaz.

Bir eşleşmeyle birden fazla sembol de gelebilir. Hoca bunu "sid" ve bir sembol örneğiyle anlattı: kaynak kod alanında "side" gelsin; "s", "i", "d" arama alanında eşleşir, "e" eşleşmez. Bu dört sembol birlikte gelir, kuyruktaki en eski dört sembolün yerine yazılır; start pointer dört ilerler, end pointer son eklenen "e"yi gösterir. Kuyruğun sonuna gelince start yeniden başa döner ve bu şekilde devam eder.

9. LZ78

9.1 Temel fikir

Aynı araştırmacılar LZ78'de kayan pencereye ihtiyaç olmadığını söyledi. Kayan pencere olmayınca arama alanı ve kaynak kod alanı da yoktur. Bunların yerine önceden karşılaşılmış string'lerden oluşan bir **sözlük** vardır. Sözlük başlangıçta boştur; boyu kullanılabilecek hafızayla sınırlıdır.

Kodlayıcı output stream'e yine bir token yazar, ama token bu kez iki alanlıdır:

- **Sıra belirleyicisi:** Çakışmanın sözlükte kaçıncı sırada olduğunu söyler.
- **Sembol.**

Input stream'den bir sembol (W) geldiğinde sözlükte daha önce gelip gelmediğine bakılır. Gelmemişse sıra belirleyicisi 0 yazılır ("benzerim daha önce gelmedi") ve sembol sözlüğe eklenir.

Varsa daha büyük bir çakışma olur mu diye bir sonraki sembol (K) alınır, W'nin arkasına eklenir ve sözlükte WK var mı diye bakılır. En uzun eşleşme bulununcaya kadar bu sürer. Token'ın ilk alanına en uzun çakışmanın sırası, ikinci alanına eklenen son sembol yazılır.

Sözlüğün sırası 0'dan başlar ve 0. sıra null'dur. Bazı uygulamalarda sözlüğün boyutuna kısıt getirilir; örneğin 50 sembolle sınırlıysa 0'dan 49'a kadar sıra olur, 51. string eklenemez ve geri kalanlar karakter karakter kodlanır. Bazen kısıt verilmez; sözlük, eldeki bellek ne kadarsa o kadar uzayabilir.

9.2 Örnek (ekrandan, 01:20:00)

Input: "sir␣sid␣eastman␣easily␣teases␣sea␣sick␣seals". Slayttaki tablo:

Sıra	Sözlük	Token	Sıra	Sözlük	Token
0	null				
1	"s"	(0,"s")	8	"a"	(0,"a")
2	"i"	(0,"i")	9	"st"	(1,"t")
3	"r"	(0,"r")	10	"m"	(0,"m")
4	"␣"	(0,"␣")	11	"an"	(8,"n")
5	"si"	(1,"i")	12	"␣ea"	(7,"a")
6	"d"	(0,"d")	13	"sil"	(5,"l")
7	"␣e"	(4,"e")	14	"y"	(0,"y")

Hocanın anlatımıyla:

- "s" sözlükte yoktur; 1. sıraya eklenir, token (0, "s").
- "i", "r" ve boşluk da sözlükte yoktur; 2, 3 ve 4. sıralara eklenir, token'lar (0, "i"), (0, "r"), (0, "␣").
- "s" sözlükte vardır, ama daha uzun eşleşme aranır: WK = "si" sözlükte yoktur. "si" 5. sıraya yazılır. "s" 1. sırada olduğu için token (1, "i").
- "d" yoktur: 6. sıra, (0, "d").
- Boşluk vardır, "␣e" yoktur. "␣e" 7. sıraya yazılır; boşluk 4. sırada olduğu için token (4, "e").
- "a" yoktur: 8. sıra, (0, "a"). Bundan sonra eşleşmeler yavaş yavaş üçlü olmaya başlar.
- "s" vardır, "st" yoktur: 9. sıra, (1, "t"). "m" yoktur: (0, "m"). "a" 8. sıradadır, "an" yoktur: (8, "n").
- Boşluk vardır, "␣e" de vardır (7. sıra), "␣ea" yoktur: 12. sıra, (7, "a").
- "s" vardır, "si" vardır (5. sıra), "sil" yoktur: 13. sıra, (5, "l"). "y" yoktur: (0, "y").

Tablo python ile doğrulandı; dizinin devamında 26. sıra "s" + eof'tur.

LZ77 ile karşılaştırınca yapı birbirine benzer: burada offset yerine sözlükteki sıra vardır ve length alanı yoktur, yani kaç sembolün çakıştığı söylenmez.

9.3 Kod çözme

Kod çözücü output stream'den okudukça hem metni hem sözlüğü yeniden kurar. Sözlük null ile başlar. (0, "s") okununca "s" yazılır ve sözlüğe eklenir. (0, "i") ile "i", (0, "r") ile "r", (0, "␣") ile boşluk yazılır ve sözlüğe eklenir. (1, "i") okununca sözlüğün 1. sırasına gidilir, oradaki "s"nin arkasına "i" eklenir ve "si" yazılır. (0, "d") ile "d" eklenir. (4, "e") okununca 4. sıradaki boşluğun arkasına "e" eklenir. Kod çözücü sözlüğünü aynı anda oluşturur ve input stream'i yeniden elde eder.

LZ78, LZ77'ye göre daha hızlı çalışır; sıkıştırma performansları birbirine çok yakındır.

10. Trie Ağacı ve Sözlüğün Dolması

10.1 Sözlükte arama (ekrandan, 01:30:00)

Kodlayıcı "sil" string'ini okuyunca sözlükte olup olmadığını bulmalıdır. Satır satır "burada sil var mı?" diye bakılırsa 13 kez kontrol yapılır ve ancak 13. satırda bulunur. Bu oldukça zahmetlidir. Birbirine bağlı olarak üretilmiş bu verilere ("s" arkasına "i", "si" arkasına "l" gibi)

erişmek için **trie** ağacı kullanılır; slayta göre sözlük için en iyi veri yapısıdır. Hoca bu ağaçları daha önce (PPM'de) anlattığını hatırlattı.

Trie'nin kökü null'dur. Tek karakterler kökün hemen altına, geliş sıralarıyla eklenir: 1-s, 2-i, 3-r, 4-␣, 6-d, 8-a gibi. Her düğümün altında, karşılaşılmışsa alfabedeki bütün karakterler ve eof bulunabilir. Slayttaki ağaçta örneğin "s" düğümünün altında 5-i, 9-t ve 26-eof; "i"nin (5) altında 13-l vardır. "sil" aranırken kökten "s"ye, "s"den "i"ye, "i"den "l"ye inilir; üç adımda 13. sıraya ulaşılır.

10.2 Sözlük dolunca

Slayta göre LZ78'de sözlüğe sembol ekleme işlemine sözlük dolana kadar devam edilir. Sözlük için ayrılan hafıza ya da ağacın izin verilen seviye sayısı (hoca örneğin 16 dedi) dolunca üç seçenek vardır:

- Gelen semboller sözlüğe eklenmeden kodlanır. Buna **FROZEN** (dondurma) denir.
- Bütün ağaç silinir ve yeni, boş bir ağaçla işleme kalınan yerden devam edilir.
- En az kullanılan string sözlükten çıkarılıp yerine yenisi eklenir. Bunun için her düğüme, oraya kaç kez uğrandığını sayan bir sayaç konur; sözlük dolunca örneğin üç kez kullanılmış bir string silinir ve yeni string o sıraya yerleştirilir.

Ağacın silinip yeniden kurulması ya da en az kullanılanın çıkarılması algoritmayı karmaşıklaştırır, ama türevleri ortaya çıkaran da bu durumlardır.

Hoca bir maliyet örneği de verdi: bir sembol ilk kez tek başına geldiğinde de sıra alanı ve sembol birlikte yazılır. Sözlüğün boyu örneğin 500 bin ise sıra alanı tek başına bir karakterden çok daha fazla bit tutar; oysa karakter 8 bitle yazılabilirdi. Bu durum özellikle başlangıçta olur. Dondurma da çok uzun dokümanlarda, örneğin metne başka dillerden kelimeler karıştığında, sözlüğü zorlayan özel durumlarda karşımıza çıkar.

11. LZW

11.1 Temel fikir

Welch'in katılmasıyla "olayın rengi çok değişir". LZW, LZ77 ve LZ78'den, hatta onların türevlerinden çok daha iyi performans verir; sözlük kullanan yöntem denince genelde herkesin aklına LZW gelir. Geniş bir kodlama alanına sahiptir.

- LZW'de bir **codebook** kullanılır. Bu aslında sözlüktür, sadece "daha afilli" bir isim verilmiştir. Codebook'ta yer alan her string'e **code word** denir.
- Codebook dinamik olarak oluşturulur. Kodların boyu codebook'un uzunluğuna bağlıdır; codebook'un boyu 8 ise yalnız 8 code word oluşturulur.
- Codebook'un boyu bir parametredir: 10.000 ile ya da 64 ile sınırlı olabilir, ya da donanım izin verdiği sürece sınırsız olabilir.
- Codebook'un başlangıcı alfabeden oluşur. 26 karakterlik bir alfabe kullanılıyorsa ilk 26 girdi a'dan z'ye karakterlerdir; sonrası input'a bağlı olarak ikili, üçlü string'lerle dolar.

11.2 Örnek: sözlüğün kurulması (ekrandan, 01:40:00 ve 01:50:00)

Input stream: "abbbaacbbabbaacacaabb", alfabe {a, b, c}. Tablodaki ikinci sütun, sıraların 4 bitlik ikili karşılıklarıdır (0000, 0001, …). Sözlüğe önce a (0), b (1) ve c (2) tek başına yerleştirilir.

Kural şudur: sözlükte olan en uzun string bulunur; bir sonraki sembol eklenince sözlükte yoksa bu yeni string sözlüğe eklenir, en uzun eşleşmenin sırası output stream'e yazılır ve kodlama eklenen son sembolden devam eder.

okunan	sözlükte var?	çıktı	yeni girdi
a	a var, ab yok	0	3 ab
b	b var, bb yok	1	4 bb
bb	bb var, bba yok	4	5 bba
a	a var, aa yok	0	6 aa
a	a var, ac yok	0	7 ac

```
c          c var, cb yok           2      8  cb
bba        bba var, bbab yok       5      9  bbab
bba        bba var, bbaa yok       5      10 bbaa
ac         ac var, aca yok         7      11 aca
aca        aca var, acaa yok       11     12 acaa
ab         ab var, abb yok         3      13 abb
b          (input bitti)           1
```

Output stream: **0 1 4 0 0 2 5 5 7 11 3 1** (ekrandan, 01:50:00). Sözlük ve çıktı python ile doğrulandı.

Hoca önce yalnız sözlüğün nasıl çıkarıldığını gösterdi, sonra aynı örnekte output stream'i yazdı: her adımda sözlüğe eklemeden önceki çakışmanın sırası output stream'e yazılır. Hem sözlük oluşturulur hem de çıktı üretilir.

Soru: Metnin boyutu sözlüğün boyutuna yetmezse ne olur? **Cevap:** Bu yöntemde bazen codebook'un boyutu verilir; "100 code word'lük bir codebook oluştur" denirse 100. satırda durulur. Ya da sözlük, hafızanın izin verdiği yere kadar (örneğin 20 bin) büyütülür. Sözlük dolduktan sonra ekleme yapılmaz ve aşağıda gösterilen yoldan devam edilir.

11.3 Sınırlı codebook (ekrandan, 01:54:40)

Hoca aynı örneği sınırlı bir codebook ile yeniden çözdü. Önce a, b, c yerleştirilir, sonra sözlük ab (3), bb (4), bba (5) ve aa (6) eklenince dolar:

- a var, ab yok → ab eklenir, çıktı 0.
- b var, bb yok → bb eklenir, çıktı 1.
- bb var, bba yok → bba eklenir, çıktı 4.
- a var, aa yok → aa eklenir, çıktı 0. Bundan sonra sözlüğe hiçbir ekleme yapılamaz.

Sözlük dolduktan sonra her adımda sözlükte var olan en uzun eşleşmenin sırası yazılır ve eşleşmeyen sembolden devam edilir:

- a var, ac yok; eklenemez → a yazılır, 0. c'den devam edilir.
- c var, cb yok → 2. b'den devam edilir.
- b, bb, bba var; bbab yok → bba, 5. b'den devam edilir.
- b, bb, bba var; bbaa yok → bba, 5.
- a var, ac yok → 0. c var, ca yok → 2. a var, ac yok → 0. c'den devam edilir.

Hocanın tahtaya yazdığı çıktı "0 1 4 0 0 2 5 5" ve "0 2 0" ile buraya kadardır. Kalan kısım python ile tamamlandı: c → 2, aa → 6, bb → 4. Sonuç: **0 1 4 0 0 2 5 5 0 2 0 2 6 4**.

Sözlük dolduktan sonra, sözlüğe ekleme için yapılan "son eklenen sembolden başla" işlemi de yapılmaz; doğrudan en uzun çakışmanın indisi yazılır.

11.4 Kod çözme (ekrandan, 01:56:30)

Açarken yine alfabeyle başlayan bir sözlük vardır. Sözlüğün boyu sınırlıysa bu boy (örnekte 13) başlık bilgisinde yazılır; sınırsızsa sınırsız bırakılır. Kod çözücü 0 okuyunca "a" yazar; sonra 1 okur, "b" yazar ve "ab"yi sözlüğe ekler. Sonra 4 okur; 4 "bb"dir, "bb" yazılır. Ardından 0 ("a") gelir; bu "a" ile önceki "bb" birleşerek "bba" sözlüğe eklenir. Bir sıfır daha gelir, "a" yazılır ve "aa" sözlüğe eklenir. Böylece kod çözücü, açtıkça sözlükte olmayan yeni durumları sözlüğe ekler.

> **Ek bilgi (derste anlatılmadı):** LZW kod çözücüsü 4 kodunu okuduğunda sözlüğünde henüz yalnız 0-3 sıraları vardır; 4 numaralı girdi kodlayıcıda tam o adımda eklenmiştir. Bu özel durumda girdi, önceki string ile onun ilk sembolünün birleşimidir ("b" + "b" = "bb"). Aynı durum bu örnekte 11 ("aca") kodunda da ortaya çıkar. Python ile yazılan kod çözücü bu kuralla "abbbaacbbabbaacacaabb" dizisini eksiksiz geri verdi.

Trie ağacı burada da işi kolaylaştırır: "aac" sözlükte var mı diye bakmak için ağaçta "a"dan sonra "c" düğümü aranır; yoksa yeni düğüm açılır ve sözlükteki sırası yazılır. Hoca LZW'de sıkıştırmanın daha hızlı, açmanın biraz daha ağır olduğunu söyledi.

11.5 Sözlük yöntemlerinin toparlanması

Sözlük kullanan yöntemlerde ya LZ'nin türevleri ya da daha başarılı olduğu için LZW kullanılır; bunun dışında bir sözlük yöntemine pek gerek yoktur.

> **Hocanın vurgusu:** Kimse "LZW ile sıkıştırdım" ya da "Huffman ile sıkıştırdım" demez. Ticari ürün olarak kullandığımız sıkıştırma algoritmalarının içinde, derste görülen algoritmalar arka arkaya kullanılır.

12. Görüntü Sıkıştırmaya Giriş ve PNG

Hoca, slaytta "görüntü sıkıştırma" yazsa da çok farklı bir şey anlatmayacağını söyledi: görüntüde renkler, metinde karakterler alınır ve aynı yöntemler kullanılır. Tek fark, görüntüde kayıplı sıkıştırmaya izin olmasıdır; gözün algılamadığı renkler yok edilebilir.

Slayttaki ayrım şöyledir (ekrandan, 02:17:30):

- **Kayıpsız:** Kayıpsız sıkıştırma genellikle pikseller arası korelasyonun düşük olduğu şekil gibi görüntülerde kullanılır.
- **Kayıplı:** Fotoğraf gibi karmaşık ve yüksek korelasyona sahip görüntülerde kullanılır.

Hoca bunu şöyle açıkladı: pikseller arası korelasyon düşükse bir rengin eksilmesi resmin bozulduğunu belli eder. Korelasyon yüksekse göz, o rengin orada olmadığını algılamaz.

Kayıpsız görüntü sıkıştırma olasılık ya da sözlük tabanlı bir yöntemle yapılabileceği gibi, ikisini bir arada kullanan bir yöntemle de yapılabilir:

- **PNG (Portable Network Graphics):** LZ77 ve Huffman sıkıştırma algoritmalarını bir arada kullanan DEFLATE yöntemini kullanır. Derste ikisi de görüldü.
- **JPEG-LS ve JBIG:** Özellikle kayıpsız görüntü sıkıştırmak için tasarlanmış yöntemlerdir. JPEG'in kayıplı ve kayıpsız türleri vardır.

Aynı yöntemler, verinin içeriği farklı da olsa tek başına ya da arka arkaya kullanılabilir.

13. Run-Length, Faks Standartları ve Arşivleyiciler

13.1 Run-length

Run-length coding'in birçok versiyonu görülmüştü. Arka arkaya tekrar eden karakterlerin örüntüsünden yararlandığı için metinde iyi sıkıştırma vermez, ama görüntüde başarılıdır; resim siyah-beyaz ya da gri tonlu ise başarısı yüksektir. Renkli resimlerde her kanal ayrı ayrı ele alınmalıdır. PCX ve TIFF görüntü formatları içinde RLE kullanır.

Daha önce görülen RLE versiyonları hatırlatıldı: tek bitlik renk (siyah-beyaz) varsa tek bit kullanılır; siyah-beyaz dizilerde her tarama satırının (scan line) başına bir "dummy" beyaz konur; bir versiyonda da sembol arka arkaya üç kez tekrar ettikten sonra run değeri verilir.

13.2 Faks (ekrandan, 02:20:00)

Slayttaki bilgiler:

- Faks makinelerinde kullanılan CCITT T4 (Group 3) standardı RLE ve Huffman kodlamasını bir arada kullanır.
- İlk piksel her zaman beyaz kabul edilir. Tekrar sayıları çok farklı sayıda olasılığa sahip olduğu için 64'ten büyük tekrar sayıları önce 64'e bölünür; bölüm ve kalan farklı Huffman tablolarıyla kodlanır. Hoca bunu 0-63 arası run değerleri için termination kodları, 64'ün katları için ayrı kodlar olarak hatırlattı.
- Bu kodlama Modified Huffman (MH) ya da Group 3 One-Dimensional (G31D) olarak bilinir.
- CCITT T4 standardı, IBM tarafından 1979'da geliştirilen MMR (Modified Modified READ) sıkıştırmasını da kullanabilir; Group 3 Two-Dimensional (G32D) olarak bilinir. Bu yöntemde bir önceki satır dikkate alınarak (iki satır birbiriyle karşılaştırılarak) daha yüksek sıkıştırma oranlarına ulaşılır.
- MMR'nin sıkıştırma performansı daha sonra geliştirilen JBIG standardının gerisinde kalmıştır. JBIG daha karmaşıktır ama daha başarılı sonuç verir.

Ek bilgi (derste anlatılmadı): Kaynaklarda Group 3'ün iki boyutlu kodlaması genellikle MR (Modified READ) olarak, MMR ise Group 4 faks (CCITT T.6) standardının kodlaması olarak geçer. Slayttaki "T4 MMR'yi kullanabilir" ifadesi büyük olasılıkla bu iki kodlamayı birlikte anıyor.

13.3 ZIP, gzip ve diğer arşivleyiciler

Günlük hayatta ZIP ve gzip kullanılır. Bunlar LZ77'nin bir türeviyle statik Huffman kodlamayı birlikte kullanır. Metin bloklara bölünür (hoca 128 ya da 256 byte'lık bloklar örneğini verdi) ve her blok ayrı bir Huffman ağacıyla sıkıştırılabilir. ZIP ve gzip'te kayan sözlük 32 KB, kaynak alanı 258 byte olacak şekilde ayarlanmıştır.

PKZIP, ARC gibi başka arşivleyiciler de vardır. Hoca bunların kendi içinde farklı birleşimler kullandığını anlattı: biri LZW ile Huffman'ı, ARC dinamik LZW ile RLE'yi, bir diğeri LZW ile Shannon-Fano'yu kullanır. Aynı yöntemler farklı kombinasyonlarla farklı sıkıştırma formatlarında karşımıza çıkar. Bir klasörü zipleyince içinde hem resim hem metin dosyası olabilir; metin ve resim için ayrı algoritma kullanılmaz, aynı yöntem farklı başarılar elde eder.

14. JPEG-LS Tahmini

Hoca, JPEG'e "LS" takısı gelince kayıpsız (lossless) JPEG olduğunu söyledi ve slayttaki tahmin yöntemini anlattı (ekrandan, 02:27:50). Kodlanan pikselin batısındaki (W), kuzeyindeki (N), kuzeybatısındaki (NW) ve kuzeydoğusundaki (NE) komşulara bakılır. Bu komşuların renk değerlerine bakılarak piksel için bir G tahmini yapılır. Slayttaki formül:

$$G = \begin{cases} \min(W, N) & \max(N, W) \le NW \\ \max(W, N) & \min(N, W) < NW \text{ (slayttaki yazım)} \\ W + N - NW & \text{diğer durumlarda} \end{cases}$$

Hoca bunu şöyle okudu: kuzeybatıdaki piksel değeri batı ve kuzeyin maksimumundan büyük ya da eşitse batı ve kuzeyin minimumu alınır; minimumdan küçükse maksimum alınır; diğer durumlarda kuzey ile batı toplanıp kuzeybatı çıkarılır. Komşu piksellere yakın değerler verildiğinde aynı renk koduna sahip pikseller artar, kullanılan renk sayısı azalır ve sıkıştırma performansı yükselir.

Ders sonrası not: Slayttaki ikinci koşul "$\min(N, W) < NW$" yazılı. Hocanın sözlü okuması ("minimumdan küçükse maksimumu al") ve yöntemin işleyişi bu koşulun $NW \le \min(N, W)$ olduğunu gösteriyor; slayttaki yazım büyük olasılıkla hatalıdır. Bu tahmin edicide NW, W ve N'nin ikisinden de büyükse küçüğü, ikisinden de küçükse büyüğü seçilir.

Slayttaki diğer bilgiler: tahmin iki aşamada gerçekleşir. İlk iki madde dikey ve yatay kenarları yakalar; örneğin $N > W$ ve $N \le NW$ ise bu yatay bir kenarı işaret eder ve W tahmin olarak kullanılır. Elde edilen G tahmininden sonra ikinci bir geçişle, tahminin çevresindeki piksel çiftlerinin farklarına göre düzeltme yapılır. Üç piksel çifti kullanılır: (NW, W), (NW, N) ve (N, NE). İki bitişik pikselin farkına göre her biri 9 gruptan birine sınıflandırılır; bu toplamda $9^3 = 729$ farklı durum verir. Her durum, tahmini düzeltmek için kendi düzeltme değerini ve o durumdaki önceki tahminlerin kalitesiyle ilgili bilgiyi saklar. Algoritma ikinci geçişten sonra sonuç tahmini üretir ve gerçek değer ile arasındaki farkı kodlar.

15. JPEG

15.1 Genel özellikler (ekrandan, 02:30:00)

Hoca JPEG'i IEC (International Electrotechnical Commission) tarafından geliştirilmiş bir standart olarak tanıttı; kayıplı ve kayıpsız olarak kullanılabilir. Hem renkli hem gri tonlu resimlerde kullanılır; siyah-beyaz bir resimde JPEG kullanmanın anlamı yoktur, yalnız maliyeti artırır. Renkli resimlerde her kanalda ayrı ayrı çalışılır ve resim her zaman sol üst pikselden başlanarak taranır. Bir bitmap JPEG'e çevrilirken kullanıcıya kayıp oranı için seçenek sunulur.

Ders sonrası not: JPEG adı Joint Photographic Experts Group'tan gelir; standart büyük olasılıkla ISO/IEC ile ITU-T'nin ortak çalışmasıdır, IEC tek başına geliştirmemiştir.

Slayttaki özellikler:

- DCT (Ayrık Kosinüs Dönüşümü) tabanlı bir standarttır.
- Sıkıştırmadaki kayıp oranı (sıkıştırma miktarı), kalite faktörünün (Q faktörü) değiştirilmesiyle ayarlanabilir. Bunlar, önceden resimler üzerinde çalışılarak çıkarılmış hazır matrislerdir; "%20 sıkıştırma için bunu, %50 için şunu kullan" denir.
- 24 bit renk derinliğine sahip fotoğraflarda (3 kanallı resimlerde) çok iyi sonuçlar verir; sayısal fotoğrafçılıkta kullanılır.
- Birçok görüntü sıkıştırma yönteminin bir araya getirilmesiyle oluşturulmuştur.
- Sıkıştırma ve açma süreleri, tek algoritmaya sahip kayıpsız yöntemlere göre daha uzundur.

Ders sonrası not: Hoca JPEG'in seçenek sunduğunu anlatırken bir yerde "kayıpsız bir sıkıştırma yapıyor, oranı %10-20 seçtin" dedi. Oran seçilerek yapılan, kayıp kabul edilen sıkıştırma kayıplı sıkıştırmadır; büyük olasılıkla "kayıplı" demek istedi.

JPEG mimarisinin adımları şunlardır: renk dönüştürme, örneklemeyi azaltma, 8×8 bloklara DCT, niceleme, zigzag tarama, RLE ve Huffman. Açarken bu adımlar tersten uygulanır.

15.2 Renk dönüştürme (ekrandan, 02:35:30)

Kullanılan resimler genelde Red, Green, Blue kanallarından oluşur. JPEG'de RGB'de de çalışılabilir, ama istenirse RGB, YUV renk formatına dönüştürülebilir. Slayttaki bilgiler:

- RGB'den YUV'a dönüştürme JPEG sıkıştırması için gerekli değildir, ama bu dönüşüm sayesinde daha yüksek sıkıştırma oranlarına ulaşılır. R, G ve B bileşenleri arasındaki korelasyonu azalttığı için sıkıştırmanın etkinliği artar.
- YUV renk formatı renkleri parlaklık (luminance) ve renklilik (chrominance) olarak ifade eder. Y parlaklığı, U ve V değerlerinin kombinasyonları renkliliği ifade eder.
- RGB → YUV dönüşümü doğrusaldır:

$$
\begin{aligned}
Y &= 0{,}299R + 0{,}587G + 0{,}114B \\
U &= -0{,}169R - 0{,}331G + 0{,}500B \\
V &= 0{,}500R - 0{,}419G - 0{,}081B
\end{aligned}
$$

R, G ve B 0 ile 255 arasında değer alır. Y parlaklık hakkında, U ve V renkler hakkında bilgi verir. Başka renk paletleri ve aralarındaki geçiş formülleri de vardır; hoca bunları ezbere bilmek gerekmediğini söyledi.

Ders sonrası not: Slaytta V satırındaki G katsayısı "0,149" gibi okunuyor. Gri bir piksel ($R = G = B$) için renklilik 0 olmalıdır, yani U ve V'nin katsayıları toplamı 0 olmalıdır. $0{,}500 - 0{,}419 - 0{,}081 = 0$ olduğu için doğru katsayı büyük olasılıkla $-0{,}419$'dur (toplamlar python ile kontrol edildi).

15.3 Örneklemeyi azaltma (down-sampling)

İnsan gözü parlaklığa daha çok, renkliliğe daha az hassastır. Bir resmin parlak ya da donuk olduğu net anlaşılırken birbirine yakın renkler arasındaki fark o kadar iyi ayırt edilemez. Bu yüzden U ve V değerlerini bir miktar azaltmak, görüntüyü insan gözü için çok fazla farklılaştırmaz. Slayta göre 2×2 ebatında 4 piksellik bir blok için Y, U ve V değerlerinden 4'er adet (4:4:4) kullanmak yerine 4:2:2 ya da 4:1:1 kullanılabilir; bu işleme down-sampling denir.

Ders sonrası not: Hoca anlatırken Y için de 4'ün 2'ye düştüğünü söyledi. Slayttaki gösterimde ilk sayı parlaklıktır ve 4'te kalır; azaltılan U ve V'dir (4:2:2'de ikişer, 4:1:1'de birer örnek). Gözün parlaklığa daha duyarlı olması da bunu gerektirir.

15.4 DCT (ekrandan, 02:40:00)

Slayttaki bilgiler:

- Görüntü 8×8 bloklara bölünür ve her blok diğerinden bağımsız olarak ayrık kosinüs dönüşümüne (DCT) tabi tutulur.
- Sonuçta yine 64 katsayı çıkar. Bunlardan ilkine DC (ilk piksel, $D(0,0)$), diğerlerine AC adı verilir.
- İnsan gözü yüksek frekanstaki değerleri algılayamaz; resimde sıkıştırma yapılacaksa yüksek frekanslar çıkarılır.
- Blok içindeki renk değerleri 0-255 arasındadır. DCT $(-128, 127)$ aralığında çalıştığından bütün renk değerlerinden 128 çıkarılır; böylece T matrisi elde edilir. 0'dan 128 çıkınca -128, 255'ten 128 çıkınca 127 olur.
- Her bloğa ayrı ayrı DCT uygulanır. Düşük frekanslı değerler matrisin sol üst köşesinde, yüksek frekanslı değerler sağ alt köşede yer alır.
- DCT katsayılarından oluşan matris slaytta $D = M^T T M$ olarak verilmiştir.

M matrisi şöyle tanımlanır ($N = 8$, $i, j = 0, \ldots, 7$):

$$M_{i,j} = \begin{cases} \dfrac{1}{\sqrt{N}} & i = 0 \\ \sqrt{\dfrac{2}{N}} \cos\left[\dfrac{(2j+1)i\pi}{2N}\right] & i > 0 \end{cases}$$

Hoca örnek verdi: 5. satır, 0. sütundaki eleman hesaplanacaksa satır 0 olmadığı için ikinci satırdaki formül kullanılır.

> **Ders sonrası not:** Bu M tanımında satır indisi i frekansı gösterir. Python ile denendiğinde ileri dönüşüm $D = MTM^T$ alınırsa ters dönüşüm $M^T D M$ orijinal bloğu geri verir; slayttaki $D = M^T T M$ ve ters dönüşümdeki $M^T R M$ aynı biçimde olduğu için ikisi birlikte doğru olamaz. Slayttaki ileri dönüşüm formülü büyük olasılıkla $D = MTM^T$ olmalıdır.

15.5 Niceleme (ekrandan, 02:47:00)

D matrisine önceden hazırlanmış bir quantization (niceleme) matrisi uygulanır:

$$C_{i,j} = \text{round}\left(\frac{D_{i,j}}{Q_{i,j}}\right)$$

Slayttaki bilgiler:

- DCT dönüşümü sonrasında ortaya çıkan katsayılar genellikle ondalıklıdır; niceleme yöntemiyle tamsayıya dönüştürülür.
- Nicemleme yönteminde her katsayı 1 ile 255 arasında bir tamsayıya bölünür ve en yakın tamsayıya yuvarlanır. Kayıp bu esnada meydana gelir.
- Katsayıların hangi değere bölüneceği 8×8 büyüklüğündeki niceleme tablolarıyla belirlenir. Parlaklık ve renklilik için farklı tablolar kullanılır.
- Görüntüdeki kayıp miktarını belirlemek için bir kalite çarpanı (quality factor) kullanılır; tablolar kalite çarpanına göre değişir.

Slaytta 75 kalite faktörü için parlaklık tablosu gösterildi:

```
 8   6   5   8  12  20  26  31
 6   6   7  10  13  29  30  28
 7   7   8  12  20  29  35  28
 7   9  11  15  26  44  40  31
 9  11  19  28  34  55  52  39
12  18  28  32  41  52  57  46
25  32  39  44  52  61  60  51
36  46  48  49  56  50  52  50
```

Hoca, %75 kayıp kabul edilirse bu Q matrisinin, %50 ya da %25 için başka matrislerin kullanıldığını söyledi. D, Q'ya oranlanır, ondalıklı sonuçlar yuvarlanır ve C matrisi elde edilir. Yeni resimde 8×8 renk piksellerinin yerine artık bu C konur; her 8×8 blok için ayrı bir C bulunur. Sonra bunlar Huffman ile sıkıştırılarak sıkıştırılmış resim formatı elde edilir. Hoca ekranda örnek bir resmin JPEG halinin 65 KB olduğunu gösterdi.

15.6 Zigzag, RLE ve Huffman (ekrandan, 02:50:00)

Slayttaki bilgiler:

- C matrisinin sağ alt köşesi 0 ile doludur, çünkü quantization işlemi yüksek frekansları yok eder.
- C matrisine zigzag coding uygulanır, arkasından değişken uzunluklu kodlama kullanılarak sıkıştırılır.
- Ters dönüşüm için $R_{i,j} = Q_{i,j} \cdot C_{i,j}$.
- DCT^{-1} için $N = \text{round}(M^T \times R \times M) + 128$.

Slayttaki şekilde 8×8 blok zigzag sırasıyla numaralanmıştır (0, 1, 5, 6, 14, … birinci satır); 0 numaralı hücre DC, koyu hücreler değeri 0'dan farklı AC'ler, açık hücreler değeri 0 olan AC'lerdir. Köşegenin altında kalan değerler gözün pek ayırt edemeyeceği değerlerdir; algoritma bunları sıfırlar. Zigzag okunan veriye run-length uygulanınca arka arkaya çok sayıda sıfır geldiği için çok iyi sıkıştırma olur. Hoca, resmi 32×32 ya da 16×16'lık parçalara bölüp her birine zigzag uygulama fikrini daha önce anlattığını hatırlattı. Arkasından entropi kodlaması yapılır; burada Huffman da Shannon-Fano da kullanılabilir, ama JPEG Huffman'ı kullanır.

DC katsayısı tek bir değerdir ve onun için ayrı bir işlem yapılır; hoca bunun sinyal işlemeyle ilişkili olduğunu, ayrıntısına gerek olmadığını söyledi.

> **Ek bilgi (derste anlatılmadı):** JPEG'de her bloğun DC katsayısı doğrudan değil, bir önceki bloğun DC katsayısından farkı olarak (DPCM) kodlanır. Komşu blokların ortalama parlaklıkları genellikle yakın olduğu için bu farklar küçük çıkar ve daha az bitle kodlanır.
>
> **Ders sonrası not:** Hoca bir yerde DC katsayısının "en yüksek frekans" olduğunu, ders sonunda da görüntüde "AC frekansları bırakmak, DC frekansları yok etmek" gibi işlemler yapıldığını söyledi. Slayta göre DC sol üst köşedeki katsayıdır ve en düşük (sıfır) frekansı, yani bloğun ortalama değerini gösterir; niceleme ile sıfırlanan, sağ alt köşedeki yüksek frekanslı AC katsayılarıdır (sabit bir 8×8 blokta yalnız DC katsayısının sıfırdan farklı çıktığı python ile kontrol edildi).

15.7 JPEG'i açmak

Açarken orijinal resme ya da kayıplı sıkıştırılmışsa orijinale yakın bir resme dönülür; bunun için ters işlem yapılır. Sıkıştırırken D, Q'ya bölünüp C bulunmuştu; açarken C, Q ile çarpılıp R elde edilir. Sıkıştırırken M matrislerinin arasına T konmuştu; açarken R konur. Sıkıştırırken renklerden 128 çıkarıldığı için sonuca 128 eklenerek orijinal renklere yakın değerlere dönülür. Elde edilen piksel değerleri resme konur ve ekranda gösterilir.

Kayıpsız sıkıştırma yapılıyorsa niceleme adımına gerek kalmaz; D doğrudan kullanılır.

JPEG dosyasının da bir başlığı vardır. Başlık bilgisinde kayıplı mı kayıpsız mı sıkıştırma yapıldığı, kayıplıysa hangi tablonun kullanıldığı ve Huffman'ın başlık bilgisi bulunur; altında da veri gelir. Ekranda gördüğümüz JPEG formatının arka planında bunlar saklıdır. YUV formatında çalışıldıysa en sonda resim RGB'ye çevrilerek ekrana yazdırılabilir.

16. JPEG 2000, GIF, TIFF ve PNG

16.1 JPEG 2000

JPEG 2000 de bir JPEG versiyonudur. En önemli farkı DCT yerine wavelet dönüşümünü kullanmasıdır; wavelet de sinyal işlemenin bir konusudur. JPEG 2000 sıradan kullanıcıların kullandığı bir format değildir; genelde istemci-sunucu görüntü uygulamalarında ve kaynağı sınırlı kablosuz cihazlarda kullanılır.

Ders sonrası not: Hoca JPEG 2000'in wavelet olarak "Haar" dönüşümünü kullandığını söyledi. JPEG 2000 standardı büyük olasılıkla kayıpsız mod için 5/3, kayıplı mod için 9/7 (Cohen-Daubechies-Feauveau) wavelet'lerini kullanır.

16.2 GIF (ekrandan, 02:53:30)

Slayttaki bilgiler:

- CompuServe tarafından geliştirilmiştir (1987).
- LZW tabanlı çalışan bir algoritmadır.
- En fazla 8 bit renk derinliğine ($2^8 = 256$ renk) sahip görüntülerde çalışır.
- Renkli fotoğraf görüntülerinin sıkıştırılmasında yetersiz kalır.
- Birkaç rengin çoğunlukta olduğu grafiksel gösterim, basit şekiller gibi görüntülerin kayıpsız olarak sıkıştırılmasında kullanılır.
- 87a ve 89a diye iki versiyonu vardır. 89a, tek bir GIF dosyası içinde birden çok GIF formatlı resmin yerleştirilmesine ve anime edilmesine olanak tanır.
- Katmanlı görüntü saklama özelliğine sahiptir; kullanım oranı düşüktür (patent koruması altında); donanımdan bağımsız çalışır.

Animasyonlu GIF'lerde arka arkaya bir sürü frame vardır; her frame ayrı ayrı sıkıştırılır ve ayrı ayrı açılır. Önemli olan arka planda LZW algoritmasının olmasıdır.

16.3 TIFF ve PNG

TIFF formatı Adobe tarafından geliştirilmiştir. 5.0 sürümünde LZW, 6.0 sürümünden itibaren JPEG kullanır; 24 bit derinliğindeki true color resimlerde başarılıdır.

Ders sonrası not: TIFF ilk olarak büyük olasılıkla Aldus şirketi tarafından geliştirildi; format, şirketin Adobe'ye geçmesiyle Adobe'nin oldu.

PNG (Portable Network Graphics) de kayıpsız bir resim formatıdır.

Ders sonrası not: Hoca ders sonunda PNG'de "LZW algoritması kullanılıyor" dedi. Kendi slaytı (ekrandan, 02:17:30) ve dersin başındaki anlatımı PNG'nin LZ77 ve Huffman'ı birlikte kullanan DEFLATE yöntemini kullandığını söyler; doğrusu budur.

Hocanın vurgusu: PNG, TIFF, GIF, animasyonlu GIF, JPEG gibi bütün resim formatlarının arka planında ya Huffman gibi istatistiksel yöntemler ya da LZW gibi sözlük kullanan yöntemler ve bunlar üzerinde küçük oynamalar vardır. Görüntüde bunlara ek olarak gözün algılamadığı renkleri çıkarmak, niceleme yapmak, parlaklığa daha hassas olunduğu için renklilik bilgisini azaltmak gibi ön işlemler yapılır; geri kalanında aynı yöntemler kullanılır.

Örnek Sorular

Soru 1 (ekrandan, 00:30:00): Arama alanında "sir␣sid␣eastman␣easily␣t", kaynak kod alanında "eases␣sea␣sick␣seals" varken LZ77 kodlayıcısı (en uzaktaki çakışmayı kullanan) hangi token'ı üretir?

Çözüm (derste verildi): "e" arama alanında sağdan 8. ve 16. konumlarda bulunur; ikisinde de "eas" üç sembol olarak eşleşir, dördüncü sembol eşleşmez. En uzaktaki çakışma referans alınır: **(16, 3, "e")**.

Soru 2 (ekrandan, 00:40:00 ve 00:46:10): "sir␣sid␣eastman" metnini arama alanı boşken LZ77 ile kodlayın ve token'ları çözerek metni geri elde edin.

Çözüm (derste verildi): (0,0,"s"), (0,0,"i"), (0,0,"r"), (0,0,"␣"), (4,2,"d"), (4,1,"e"), (0,0,"a"), (10,1,"t"), (0,0,"m"), (4,1,"n"). Kod çözmede her token için offset kadar geri gidilir, length kadar sembol kopyalanır ve arkasına üçüncü alan eklenir. Token'lar python ile doğrulandı.

Soru 3 (ekrandan, 00:40:00): Offset 11 bit, length 5 bit ve sembol 8 bit ise bir LZ77 token'ı kaç bittir? Token kaç karakter kodlarsa karakterleri tek tek yazmakla aynı yeri tutar?

Çözüm (derste verildi): $11 + 5 + 8 = 24$ bit. İki çakışma ve bir sembolden oluşan token 3 karakteri kodlar; bunlar orijinalde $3 \times 8 = 24$ bittir. Daha uzun eşleşmelerde kazanç başlar.

Soru 4 (ekrandan, 01:20:00): “sir␣sid␣eastman␣easily␣teases␣sea␣sick␣seals” metnini LZ78 ile kodlayın; sözlüğün ilk 14 girdisini ve token’larını yazın.

Çözüm (derste verildi): 1 “s” (0,“s”); 2 “i” (0,“i”); 3 “r” (0,“r”); 4 “␣” (0,“␣”); 5 “si” (1,“i”); 6 “d” (0,“d”); 7 “␣e” (4,“e”); 8 “a” (0,“a”); 9 “st” (1,“t”); 10 “m” (0,“m”); 11 “an” (8,“n”); 12 “␣ea” (7,“a”); 13 “sil” (5,“l”); 14 “y” (0,“y”).

> **Çözüm (derste verilmedi):** Dizinin devamı python ile: (4,“t”), (0,“e”), (8,“s”), (16,“s”), (4,“s”), (16,“a”), (19,“i”), (0,“c”), (0,“k”), (19,“e”), (8,“l”), (1,eof). Sözlük 26 girdiye ulaşır; slayttaki trie ağacıyla (örneğin 26-eof, 24-e, 21-i) uyumludur.

Soru 5 (ekrandan, 01:40:00 ve 01:50:00): Alfabe {a, b, c} ve input stream “abbbaacbbabbaacacaabb” için LZW sözlüğünü ve output stream’i bulun.

Çözüm (derste verildi): Sözlük: 3 ab, 4 bb, 5 bba, 6 aa, 7 ac, 8 cb, 9 bbab, 10 bbaa, 11 aca, 12 acaa, 13 abb. Output stream: **0 1 4 0 0 2 5 5 7 11 3 1**.

Soru 6 (ekrandan, 01:54:40): Aynı input için codebook, aa (6. sıra) eklendikten sonra dolarsa output stream ne olur?

Çözüm (derste kısmen verildi): Sözlük dolana kadar 0 1 4 0; sonrasında ekleme yapılmadan en uzun eşleşmenin sırası yazılır: 0 (a), 2 (c), 5 (bba), 5 (bba), 0 (a), 2 (c), 0 (a).

> **Çözüm (derste verilmedi):** Kalan kısım c → 2, aa → 6, bb → 4. Tam çıktı python ile: **0 1 4 0 0 2 5 5 0 2 0 2 6 4**.

Soru 7 (ekrandan, 01:56:30): LZW output stream’i “0 1 4 0 0 …” ve başlangıç sözlüğü {0 a, 1 b, 2 c} ise kod çözücü sözlüğü nasıl kurar?

Çözüm (derste verildi): 0 → “a”; 1 → “b”, sözlüğe “ab” eklenir; 4 → “bb”; 0 → “a”, sözlüğe “bba” eklenir; 0 → “a”, sözlüğe “aa” eklenir. Kod çözücü açtıkça sözlükte olmayan yeni durumları sözlüğe ekler.

> **Çözüm (derste verilmedi):** 4 okunduğunda kod çözücünün sözlüğünde henüz yalnız 0-3 vardır; 4 numaralı girdi önceki string “b” ile onun ilk sembolünün birleşimi “bb”dir. Bu kuralla yazılan python kod çözücüsü “0 1 4 0 0 2 5 5 7 11 3 1” dizisinden “abbbaacbbabbaacacaabb” metnini geri elde etti.

Dersin Toparlaması

Bu derste sözlük tabanlı yöntemler tamamlandı. LZ77, kayan bir pencerenin arama alanını sözlük olarak kullanır ve (offset, uzunluk, sonraki sembol) token’ları üretir; kod çözücüsü basit olduğu için arşiv uygulamalarına uygundur. Türevleri offset ve length alanlarıyla, buffer boylarıyla ve dairesel kuyruk gibi pencere yapılarıyla oynar. LZ78 pencereyi kaldırır, karşılaşılan string’lerden açık bir sözlük kurar ve (sıra, sembol) token’ları üretir; sözlük trie ağacında tutulur ve dolduğunda dondurma, silme ya da en az kullanılanı çıkarma seçenekleri vardır. LZW ise alfabeyle başlayan bir codebook kullanır, yalnız sözlük sıralarını yazar ve sözlük yöntemleri arasında en başarılı olanıdır.

Dersin ikinci yarısında görüntü sıkıştırmaya geçildi. Görüntüde de aynı yöntemler kullanılır; fark, gözün algılamadığı bilginin atılabilmesidir. PNG (DEFLATE), faks standartları (RLE + Huffman), ZIP/gzip, GIF ve TIFF (LZW) derste görülen yöntemlerin birleşimleridir. JPEG ise renk dönüştürme, renklilikte örneklemeyi azaltma, 8×8 DCT, niceleme, zigzag tarama, RLE ve Huffman adımlarını arka arkaya uygular; kayıp niceleme adımında oluşur. Hoca, bütün resim formatlarının arka planında istatistiksel ve sözlük tabanlı yöntemlerin ve üzerlerindeki küçük oynamaların bulunduğunu vurgulayarak dersin konularını bağladı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.